



University of Asia Pacific
Department of Computer Science and Engineering
CSE 402: Operating System Lab

Lab Report: 03

Report Title: RR CPU Scheduling and Comparison

Submitted by:

Name : Adika Sultana

Student ID : 23101162

Section : C2

Submitted to:

Atia Rahman Orthi

Lecturer,

Department of CSE, UAP

Date of Submission: 17/08/2026

1.Problem Statement:

A CPU scheduling system consists of six processes that are waiting to be executed by the processor. Each process has a specific **Arrival Time (AT)** and **Burst Time (BT)** as given in the following table:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	7
P2	1	4
P3	2	15
P4	3	11
P5	4	20
P6	4	9

For the Round Robin (RR) scheduling algorithm, the assigned Time Quantum (TQ) is 5 seconds.

The purpose of this problem is to analyze, implement, and compare the performance of different CPU scheduling techniques. The obtained results from Round Robin, FCFS, and Non-Preemptive SJF will then be compared based on their Average Turnaround Time and Average Waiting Time. To study the behavior of different CPU scheduling algorithms and determine which algorithm provides better performance in terms of minimizing waiting time and turnaround time for the given set of processes.

2. Source code:

```
from collections import deque
processes = [
    ["P1", 0, 7],
    ["P2", 1, 4],
    ["P3", 2, 15],
    ["P4", 3, 11],
    ["P5", 4, 20],
    ["P6", 4, 9]
]

def round_robin(processes, tq):

    n = len(processes)

    at = [p[1] for p in processes]
    bt = [p[2] for p in processes]
    rt = bt[:]

    ct = [0] * n
    visited = [False] * n
```

```

q = deque()

time = 0
completed = 0
sequence = []

while completed < n:

    for i in range(n):
        if at[i] <= time and not visited[i]:
            q.append(i)
            visited[i] = True

    if not q:
        time += 1
        continue

    i = q.popleft()

    sequence.append(processes[i][0])

    run = min(tq, rt[i])

    time += run
    rt[i] -= run

    for j in range(n):
        if at[j] <= time and not visited[j]:
            q.append(j)
            visited[j] = True

    if rt[i] > 0:
        q.append(i)
    else:
        ct[i] = time
        completed += 1

tat = [ct[i] - at[i] for i in range(n)]
wt = [tat[i] - bt[i] for i in range(n)]

print("\nROUND ROBIN")
print("Execution Sequence:")
print("-> ".join(sequence))

print("\n{:<8}{:<8}{:<8}{:<8}{:<8}{:<8}".format(
    "P", "AT", "BT", "CT", "TAT", "WT"))

```

```

for i in range(n):
    print("{:<8}{:<8}{:<8}{:<8}{:<8}".format(
        processes[i][0],
        at[i],
        bt[i],
        ct[i],
        tat[i],
        wt[i]
    ))

avg_tat = sum(tat) / n
avg_wt = sum(wt) / n

print("\nAverage TAT =", round(avg_tat, 2))
print("Average WT =", round(avg_wt, 2))

return round(avg_tat,2), round(avg_wt,2)

```

```

def fcfs(processes):

    p = sorted(processes, key=lambda x: x[1])

    time = 0
    result = []
    sequence = []

    for name, at, bt in p:

        if time < at:
            time = at

        sequence.append(name)

        ct = time + bt
        tat = ct - at
        wt = tat - bt

        result.append([name, at, bt, ct, tat, wt])

        time = ct

    print("\nFCFS")
    print("Execution Sequence:")
    print("-> ".join(sequence))

```

```

print("\n{:<8}{:<8}{:<8}{:<8}{:<8}{:<8}".format(
    "P", "AT", "BT", "CT", "TAT", "WT"))

for r in result:
    print("{:<8}{:<8}{:<8}{:<8}{:<8}{:<8}".format(
        r[0], r[1], r[2], r[3], r[4], r[5]
    ))

avg_tat = sum(r[4] for r in result) / len(result)
avg_wt = sum(r[5] for r in result) / len(result)

print("\nAverage TAT =", round(avg_tat,2))
print("Average WT =", round(avg_wt,2))

return round(avg_tat,2), round(avg_wt,2)

def sjf(processes):

    n = len(processes)

    completed = [False] * n

    time = 0
    done = 0

    result = []
    sequence = []

    while done < n:

        ready = []

        for i, p in enumerate(processes):
            if p[1] <= time and not completed[i]:
                ready.append((p[2], i))

        if not ready:
            time += 1
            continue

        ready.sort()

        bt, idx = ready[0]

```

```

name, at, bt = processes[idx]

sequence.append(name)

ct = time + bt
tat = ct - at
wt = tat - bt

result.append([name, at, bt, ct, tat, wt])

time = ct

completed[idx] = True
done += 1

print("\nSJF")
print("Execution Sequence:")
print("-> ".join(sequence))

print("\n{:<8}{:<8}{:<8}{:<8}{:<8}{:<8}".format(
    "P", "AT", "BT", "CT", "TAT", "WT"))

for r in result:
    print("{:<8}{:<8}{:<8}{:<8}{:<8}{:<8}".format(
        r[0], r[1], r[2], r[3], r[4], r[5]
    ))

avg_tat = sum(r[4] for r in result) / len(result)
avg_wt = sum(r[5] for r in result) / len(result)

print("\nAverage TAT =", round(avg_tat,2))
print("Average WT =", round(avg_wt,2))

return round(avg_tat,2), round(avg_wt,2)

rr_tat, rr_wt = round_robin(processes, 5)

fcfs_tat, fcfs_wt = fcfs(processes)

sjf_tat, sjf_wt = sjf(processes)

print("\nCOMPARISON")

print("{:<15}{:<15}{:<15}".format("Algorithm","Avg TAT","Avg WT"))
print("{:<15}{:<15}{:<15}".format("Round Robin",rr_tat,rr_wt))

```

```

print("{:<15}{:<15}{:<15}".format("FCFS",fcfs_tat,fcfs_wt))
print("{:<15}{:<15}{:<15}".format("SJF",sjf_tat,sjf_wt))

algorithms = [
    ("Round Robin", rr_wt),
    ("FCFS", fcfs_wt),
    ("SJF", sjf_wt)
]

best = min(algorithms, key=lambda x: x[1])

print("\nBest Scheduling Algorithm =", best[0])
print("Reason: It has the Lowest Average Waiting Time.")

```

GitHub: <https://github.com/Adika2003/CSE-402-Operating-System-Lab>

3.Result:

```

Run 23101162 LAB-4 x
C:\Users\lab\PycharmProjects\PythonProject\.venv\Scripts\python.exe "C:\Users\lab\PycharmProjects\Py

ROUND ROBIN
Execution Sequence:
P1 -> P2 -> P3 -> P4 -> P5 -> P6 -> P1 -> P3 -> P4 -> P5 -> P6 -> P3 -> P4 -> P5 -> P5

P      AT      BT      CT      TAT      WT
P1      0       7      31      31      24
P2      1       4       9       8       4
P3      2      15      55      53      38
P4      3      11      56      53      42
P5      4      20      66      62      42
P6      4       9      50      46      37

Average TAT = 42.17
Average WT  = 31.17

```

```
↑
↓
↶
↷
⇅
🖨
🗑

Average WT = 31.17

FCFS
Execution Sequence:
P1 -> P2 -> P3 -> P4 -> P5 -> P6

P      AT      BT      CT      TAT      WT
P1      0        7        7        7        0
P2      1        4       11       10        6
P3      2       15       26       24        9
P4      3       11       37       34       23
P5      4       20       57       53       33
P6      4        9       66       62       53

Average TAT = 31.67
Average WT = 20.67
```

```
↓
↶
↷
⇅
🖨
🗑

SJF
Execution Sequence:
P1 -> P2 -> P6 -> P4 -> P3 -> P5

P      AT      BT      CT      TAT      WT
P1      0        7        7        7        0
P2      1        4       11       10        6
P6      4        9       20       16        7
P4      3       11       31       28       17
P3      2       15       46       44       29
P5      4       20       66       62       42

Average TAT = 27.83
Average WT = 16.83
```

```
Average WT = 16.83
```

```
COMPARISON
```

Algorithm	Avg TAT	Avg WT
Round Robin	42.17	31.17
FCFS	31.67	20.67
SJF	27.83	16.83

```
Best Scheduling Algorithm = SJF
```

```
Reason: It has the Lowest Average Waiting Time.
```

```
Process finished with exit code 0
```

4. "SJF is better than FCFS and Round Robin because it minimizes the average waiting time and average turnaround time, resulting in better overall CPU performance"

From the calculated scheduling results, it can be concluded that the Non-Preemptive Shortest Job First (SJF) algorithm performs better than both First Come First Serve (FCFS) and Round Robin (RR) for the given set of processes. The SJF algorithm prioritizes the process with the shortest burst time, allowing shorter tasks to complete earlier and reducing the amount of time processes spend waiting in the ready queue. As a result, it provides the lowest average waiting time and average turnaround time, which improves overall CPU utilization and system performance.

Although Round Robin provides better fairness by giving each process an equal opportunity to execute and is suitable for time-sharing systems, it may increase the average waiting time due to frequent context switching. On the other hand, FCFS is simple and easy to implement but may suffer from the convoy effect, where shorter processes have to wait behind longer processes.

Therefore, based on the calculated Average TAT and Average WT, the Non-Preemptive SJF scheduling algorithm is the most efficient choice for this particular

process set, as it minimizes execution delays and provides better overall CPU scheduling performance.

5. Discussion:

The performance of Round Robin (RR), First Come First Serve (FCFS), and Non-Preemptive Shortest Job First (SJF) scheduling algorithms was evaluated based on the calculated Average Turnaround Time (Avg. TAT) and Average Waiting Time (Avg. WT). The comparison results show significant differences in the efficiency of each scheduling technique.

The Round Robin (RR) algorithm achieved an Average Turnaround Time of 42.17 seconds and an Average Waiting Time of 31.17 seconds, which are the highest among all three algorithms. Although RR provides fairness by assigning each process an equal CPU time slice of 5 seconds, the frequent context switching between processes increases execution overhead. As a result, processes take more time to complete, leading to higher waiting and turnaround times. However, RR is still suitable for interactive and time-sharing systems where fairness and quick response are more important.

The First Come First Serve (FCFS) algorithm produced an Average Turnaround Time of 31.67 seconds and an Average Waiting Time of 20.67 seconds. It performs better than Round Robin because it has less scheduling overhead and follows a simple execution order based on process arrival time. However, FCFS does not consider the burst time of processes, which can cause shorter processes to wait behind longer processes. This limitation, known as the convoy effect, reduces the overall efficiency of the CPU.

The Non-Preemptive Shortest Job First (SJF) algorithm provided the best performance with an Average Turnaround Time of 27.83 seconds and an Average Waiting Time of 16.83 seconds. Since SJF selects the process with the shortest burst time first, shorter processes complete earlier and spend less time waiting in the ready queue. This optimized execution order reduces the overall delay and improves CPU utilization.

From the comparison of all three algorithms, it is clear that SJF performs better than FCFS and Round Robin for the given set of processes. It achieves the lowest average waiting time and average turnaround time, indicating better scheduling efficiency and faster process completion. Although RR provides better fairness and

FCFS offers simplicity, SJF provides the most effective CPU performance by minimizing unnecessary waiting time.

Therefore, based on the calculated results, the Non-Preemptive SJF scheduling algorithm is considered the most suitable algorithm for this particular scenario. It ensures efficient CPU utilization, reduces process delays, and provides better overall system performance compared to the other two scheduling methods.